ANSIBLE TASK - 04

1.Create ansible playbook to create ansible vault.

Ansible Vault is used to encrypt sensitive data such as passwords, database credentials, API keys, and secret variables so they are not stored in plain text.

Since you already have an Ansible control node and two worker nodes in your lab, we can create an encrypted Vault file and use it in a playbook to configure an application on your managed servers.

Step 1: Understand the workflow

1. Create Vault file

Store username and password variables

2. Encrypt the file

Use ansible-vault create

3. Create the playbook

Load secrets using vars_files

4. Run and verify

Provide the Vault password at runtime

Step 2: Verify Ansible installation

Run these commands on your Ansible control node:

ansible --version

ansible-vault --help

If both commands work, continue to the next step.

Step 3: Create a working directory

mkdir -p ~/ansible-vault-lab

cd ~/ansible-vault-lab

Create an inventory file:

vi inventory

Add your two worker nodes, replacing the example IP addresses if your current inventory uses different ones:

[appservers]

172.31.17.140

172.31.16.206

[appservers:vars]

ansible_user=ansadm

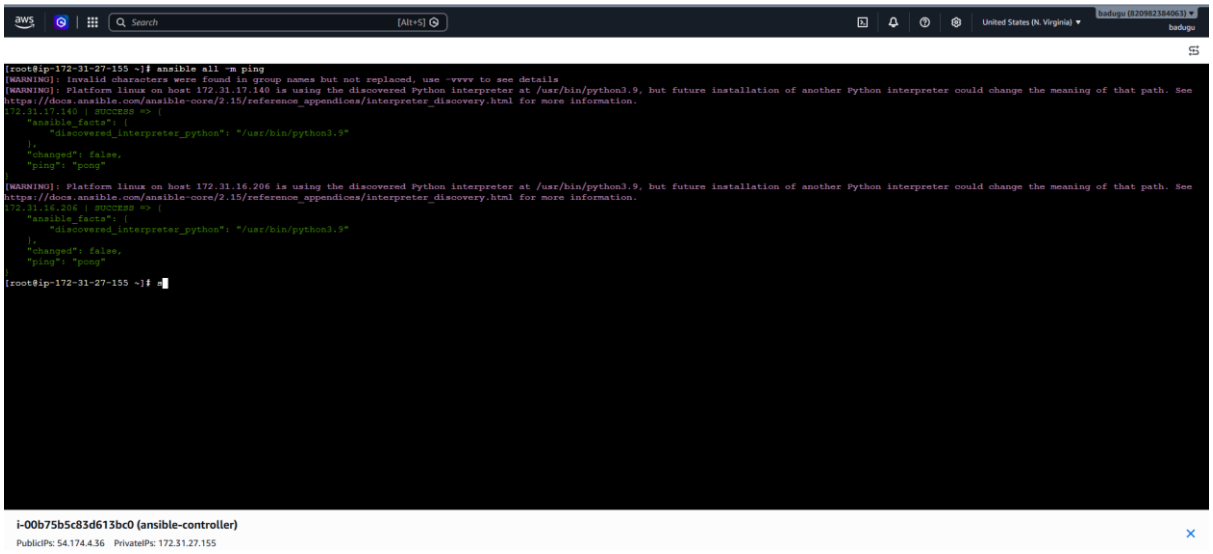
ansible_become=true

Save and exit. These are the worker IPs from your existing lab; confirm they are still current.

Test connectivity:

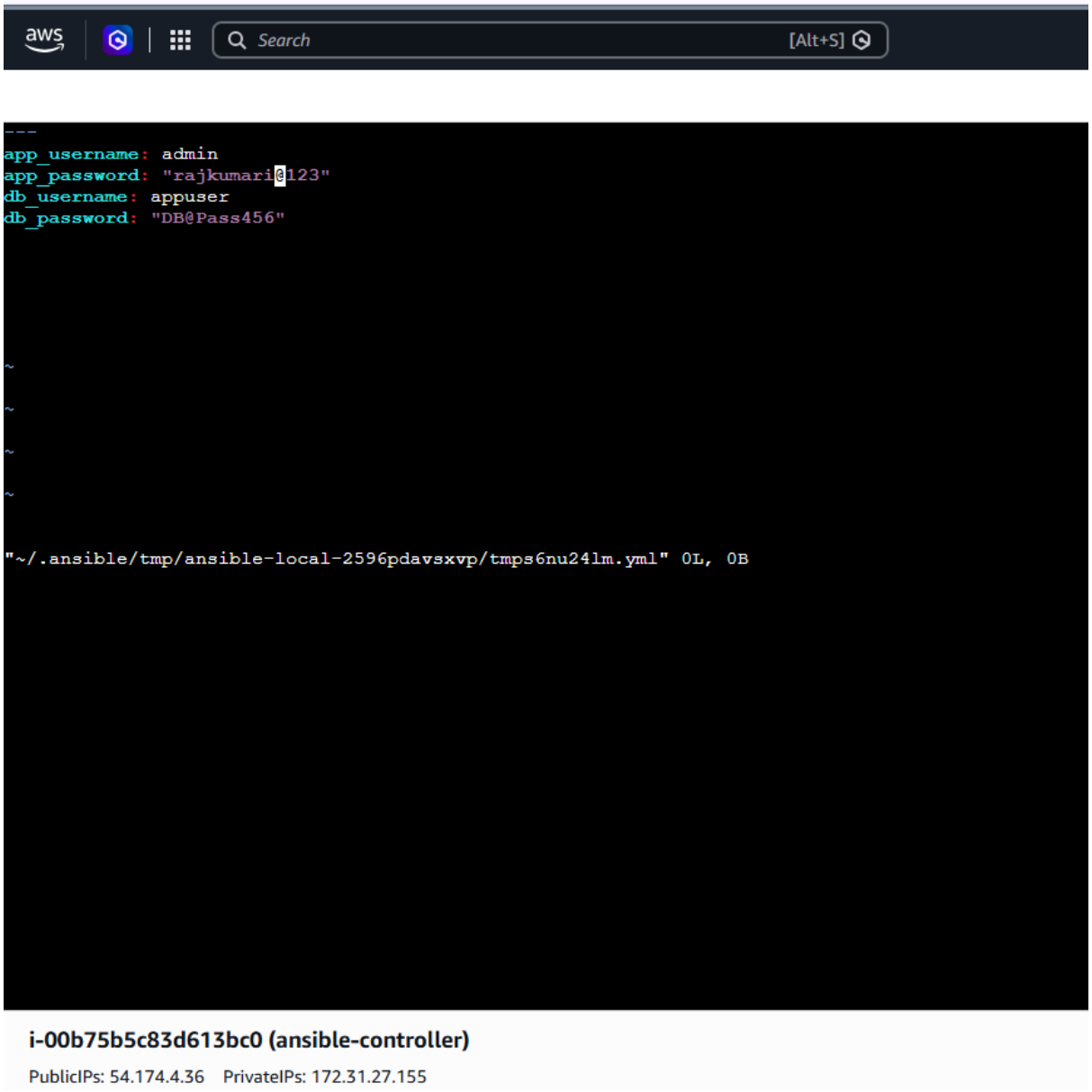
ansible -i inventory appservers -m ping

Both hosts should return "ping": "pong"



Ansible will prompt you to enter and confirm a Vault password. Choose a password for this lab and keep it safe.

Your editor will open. Add the following YAML variables:



Save and exit the editor.

- In vi, press Esc, type :wq, and press Enter.
- Ansible automatically encrypts the file when you save and exit.

Important: These are sample lab credentials only. Use securely generated credentials for real applications.

Step 5: Verify that the file is encrypted

Run:

cat vault.yml

You should see output beginning with something similar to:

$ANSIBLE_VAULT;1.1;AES256

The remaining content will look like encrypted hexadecimal data, not readable YAML.

To view the decrypted contents temporarily:

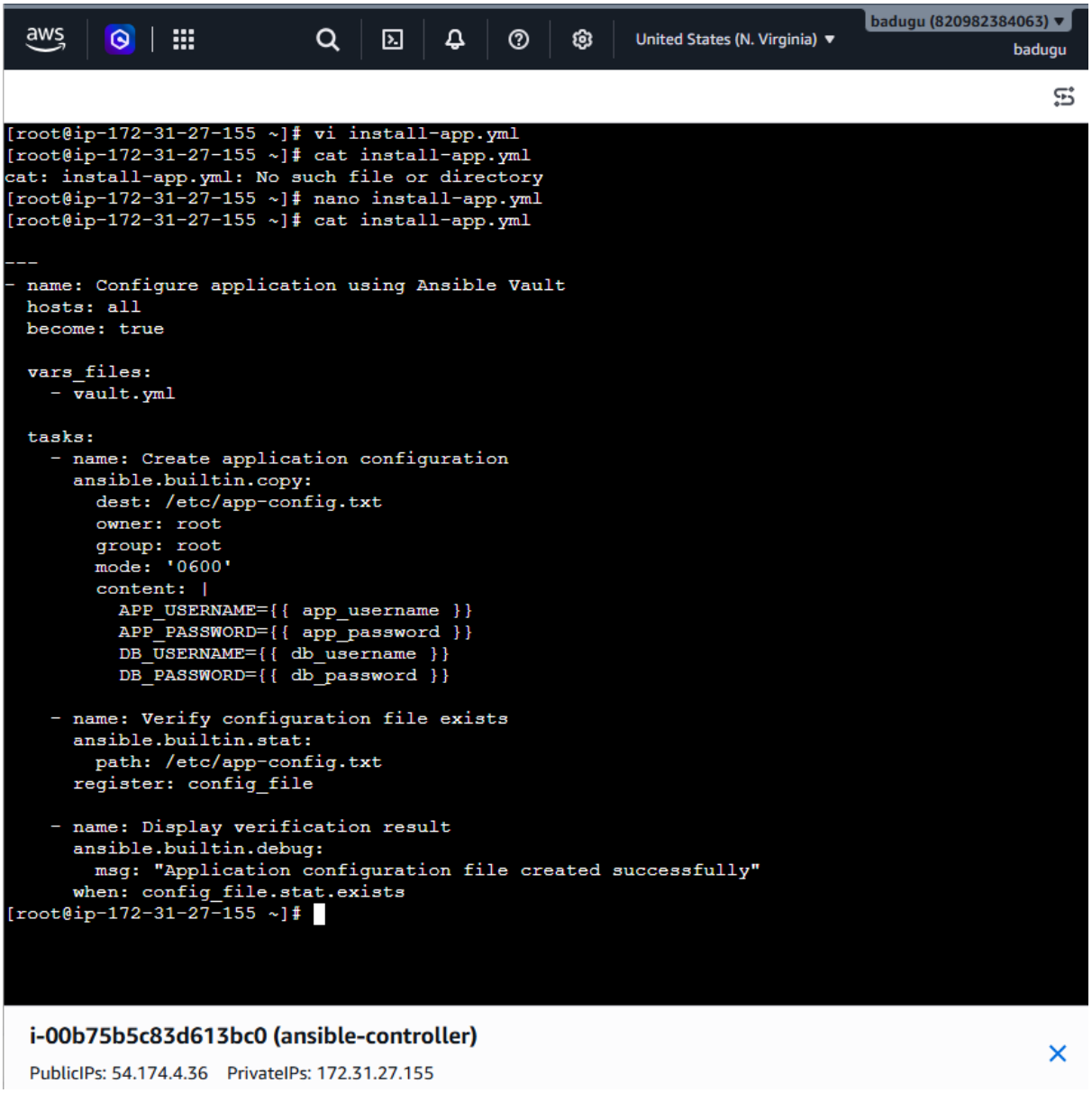
ansible-vault view vault.yml

Enter your Vault password when prompted.




```
[root@ip-172-31-27-155 ~]# ansible-vault create vault.yml
New Vault password:
Confirm New Vault password:
[root@ip-172-31-27-155 ~]# ansible-vault create vault.yml
New Vault password:
Confirm New Vault password:
ERROR! vault.yml exists, please use 'edit' instead
[root@ip-172-31-27-155 ~]# ansible-vault edit vault.yml
Vault password:
[root@ip-172-31-27-155 ~]# cat vault.yml
$ANSIBLE_VAULT;1.1;AES256
39313362353934623231306566343561613037613638333539646234303531643965363031626538
64323034323532646565396330326134376166653936363 10a6365613031656536356136373530 61
31313938316239393430623833653237343430623531613036363563303331633832613135303934
37313436313162323 80a353632303665653961363533633861623031373833306132373137326561
62326434626463356231623834323663303032656365613831353561343065366531653434373538
64373066373065383430616137653832633631306465333933633135363034393362336437323236
61626336653432636561306336363332636336623962326231643937333633653434326562393938
36313365383763636366303230333233323538356233396665313738363532316264616337333737
31336330633932306237653964616161633731623930366537663063613830326639
[root@ip-172-31-27-155 ~]#
```

```
[root@ip-172-31-27-155 ~]# ansible-vault view vault.yml
Vault password:
---
app_username: admin
app_password: "rajkumari@123"
db_username: appuser
db_password: "DB@Pass456"
[root@ip-172-31-27-155 ~]#
```



What does each section do?

Section	Purpose
hosts: appservers	Targets the two worker nodes
become: true	Runs privileged tasks using sudo
vars_files	Loads the encrypted Vault variables
copy	Creates the configuration file on each worker
mode: '0600'	Allows only the file owner to read and write it
stat	Checks whether the file exists

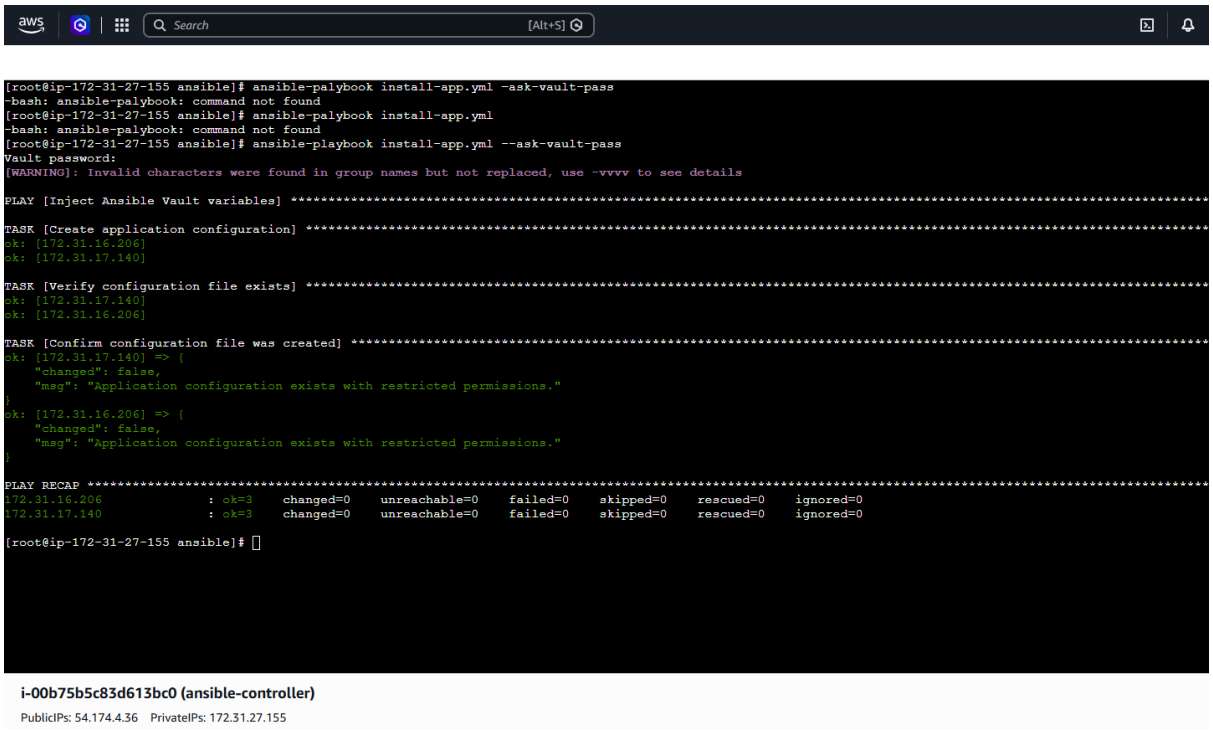
debug	Displays a success message when the file exists

The example demonstrates Vault integration; the configuration file contains decrypted credentials on the target servers. Protect it accordingly and avoid exposing secrets in logs.

Step 7: Run the playbook

From the same directory, execute:

$ ansible-palybook install-app.yml –-ask-vault-pass



Step 9: Learn the important Vault commands

Command	Purpose
ansible-vault create vault.yml	Create a new encrypted file
ansible-vault view vault.yml	View decrypted contents
ansible-vault edit vault.yml	Edit the encrypted file securely
ansible-vault encrypt secrets.yml	Encrypt an existing plaintext file
ansible-vault decrypt vault.yml	Decrypt a file to plaintext
ansible-vault rekey vault.yml	Change the Vault password

ansible-playbook -i inventory install-app.yml --ask-vault-pass	Run a playbook and prompt for the Vault password

Security note: Avoid committing plaintext secret files or storing Vault passwords in Git. In production, use a secure secret-management system or a protected CI credential to provide the Vault password.

2.Write a ansible playbook to install apache in linux and ubuntu machine by using when condition.

Install Apache using Ansible when Condition — Step by Step

We will create an Ansible playbook that installs Apache on Linux servers using a when condition to select the appropriate package based on the operating system.

For example:

- Amazon Linux / RHEL / CentOS → install httpd
- Ubuntu / Debian → install apache2

Step 1: Verify your inventory

On your Ansible control node, check your inventory:

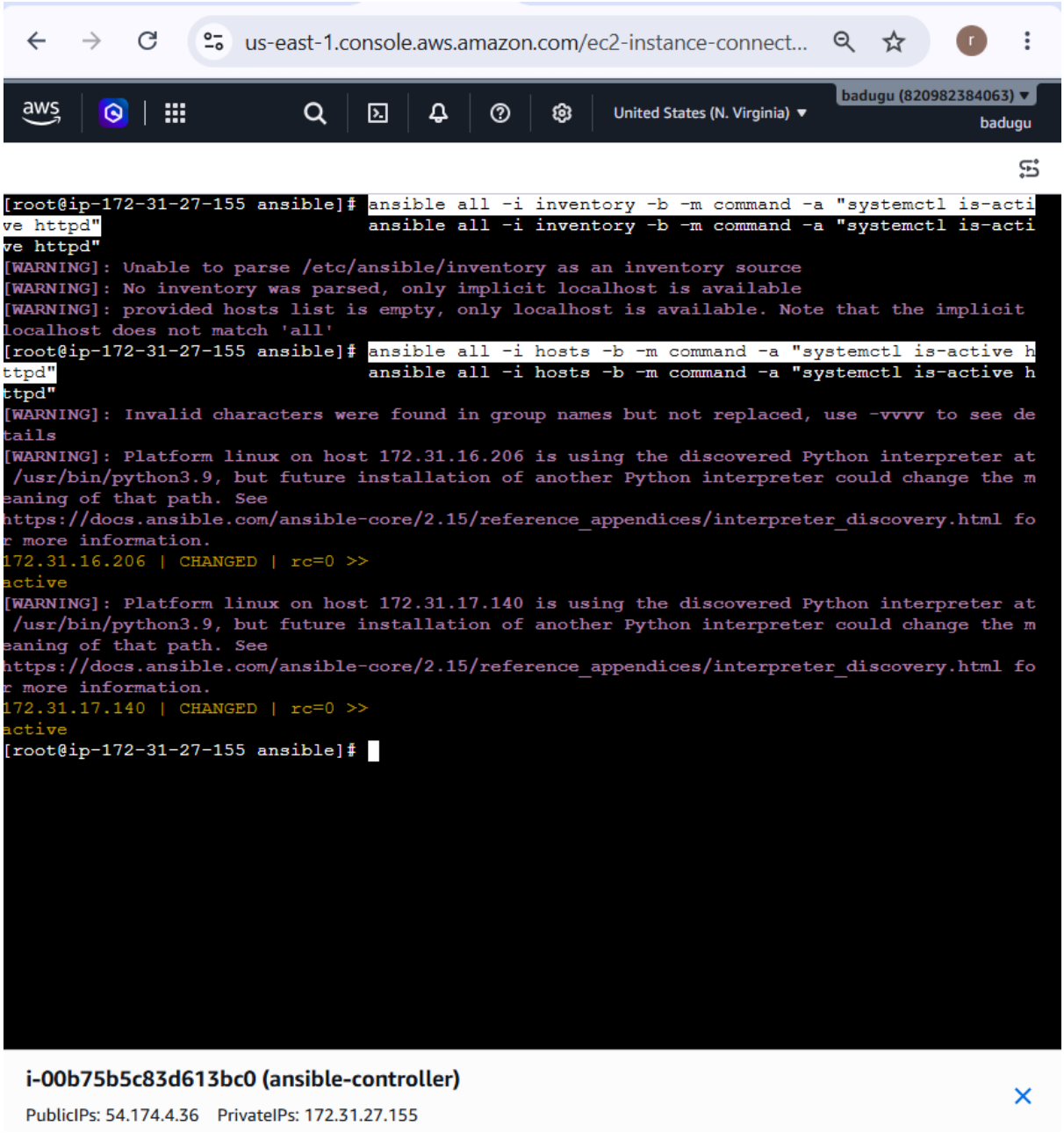
ansible-inventory --list

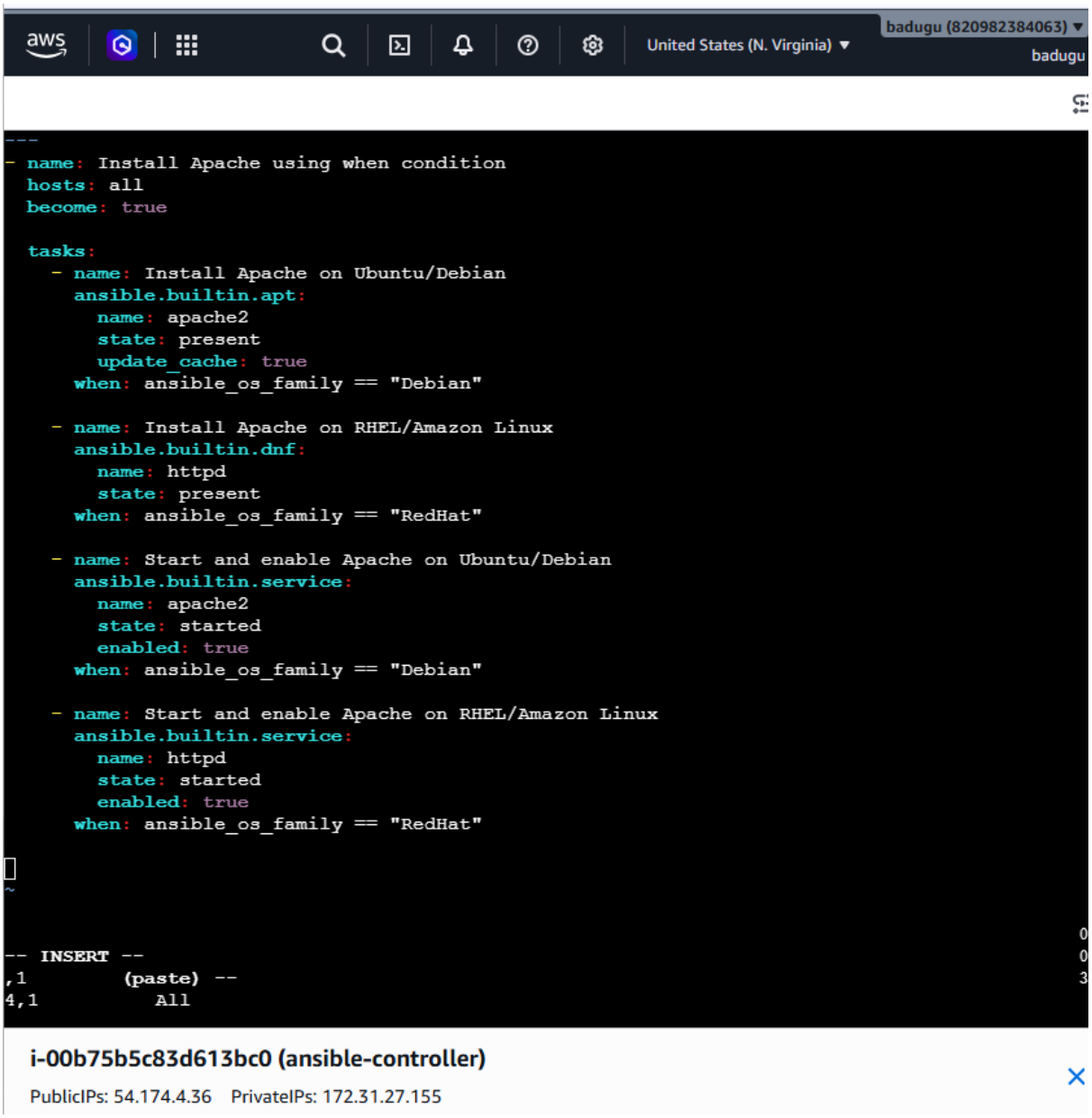
For your existing lab, you can use the all group or your worker group in the playbook.

Step 2: Create the playbook

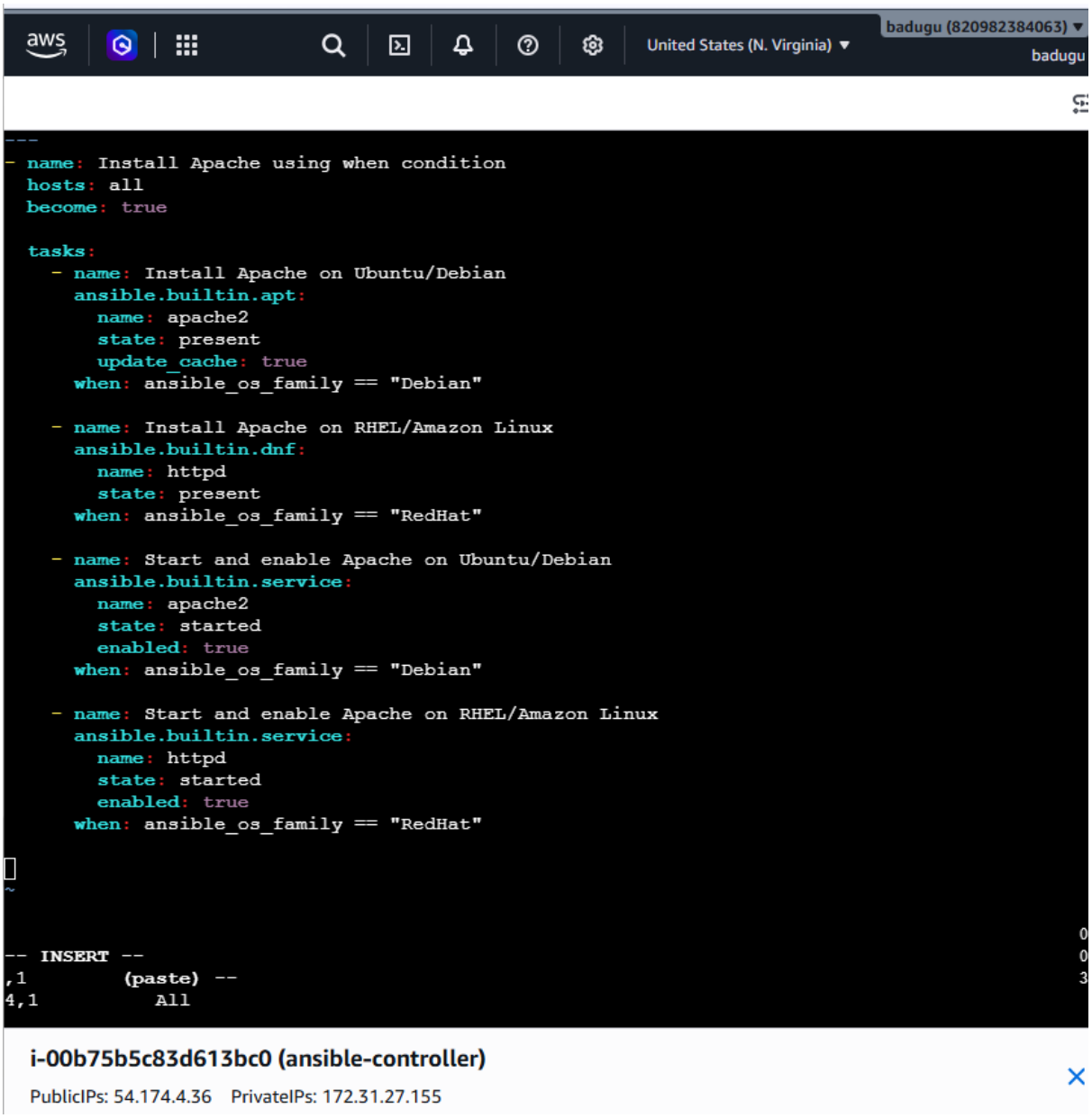
Create a file named install-apache.yml:

vi install-apache.yml


us-east-1.console.aws.amazon.com/ec2-instance-connect...
United States (N. Virginia)
badugu (820982384063)
badugu
[root@ip-172-31-27-155 ansible]# ansible all -i inventory -b -m command -a "systemctl is-active httpd"
ansible all -i inventory -b -m command -a "systemctl is-active httpd"
[WARNING]: Unable to parse /etc/ansible/inventory as an inventory source
[WARNING]: No inventory was parsed, only implicit localhost is available
[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit localhost does not match 'all'
[root@ip-172-31-27-155 ansible]# ansible all -i hosts -b -m command -a "systemctl is-active httpd"
ansible all -i hosts -b -m command -a "systemctl is-active httpd"
[WARNING]: Invalid characters were found in group names but not replaced, use -vvvv to see details
[WARNING]: Platform linux on host 172.31.16.206 is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.15/reference_appendices/interpreter_discovery.html for more information.
172.31.16.206 | CHANGED | rc=0 >>
active
[WARNING]: Platform linux on host 172.31.17.140 is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.15/reference_appendices/interpreter_discovery.html for more information.
172.31.17.140 | CHANGED | rc=0 >>
active
[root@ip-172-31-27-155 ansible]#
i-00b75b5c83d613bc0 (ansible-controller)
PublicIPs: 54.174.4.36 PrivateIPs: 172.31.27.155


aws
United States (N. Virginia)
badugu (820982384063)
badugu

- name: Install Apache using when condition
hosts: all
become: true
tasks:
- name: Install Apache on Ubuntu/Debian
ansible.builtin.apt:
name: apache2
state: present
update_cache: true
when: ansible_os_family == "Debian"
- name: Install Apache on RHEL/Amazon Linux
ansible.builtin.dnf:
name: httpd
state: present
when: ansible_os_family == "RedHat"
- name: Start and enable Apache on Ubuntu/Debian
ansible.builtin.service:
name: apache2
state: started
enabled: true
when: ansible_os_family == "Debian"
- name: Start and enable Apache on RHEL/Amazon Linux
ansible.builtin.service:
name: httpd
state: started
enabled: true
when: ansible_os_family == "RedHat"
~
-- INSERT --
,1 (paste) --
4,1 All
i-00b75b5c83d613bc0 (ansible-controller)
PublicIPs: 54.174.4.36 PrivateIPs: 172.31.27.155



Step 3: Understand the when condition

Condition	Action
ansible_os_family == "Debian"	Runs the task on Ubuntu/Debian
ansible_os_family == "RedHat"	Runs the task on RHEL-based systems, including Amazon Linux
become: true	Executes privileged operations using sudo
state: present	Ensures Apache is installed

state: started	Starts Apache if it is not running
enabled: true	Enables Apache at boot

Ansible automatically gathers facts such as ansible_os_family before running tasks, so you do not need to define that variable manually.

Step 4: Check the operating system

Run this command:

ansible all -i inventory -m setup -a 'filter=ansible_os_family'

Example output for Amazon Linux:

"ansible_os_family": "RedHat"

Step 5: Execute the playbook

ansible-playbook -i inventory install-apache.yml

If your inventory is configured in ansible.cfg, you may omit -i inventory.

Expected result:

TASK [Install Apache on RHEL/Amazon Linux]

changed: [172.31.17.140]

changed: [172.31.16.206]

TASK [Start and enable Apache on RHEL/Amazon Linux]

changed: [172.31.17.140]

changed: [172.31.16.206]

PLAY RECAP

172.31.17.140 : failed=0

172.31.16.206 : failed=0

The actual tasks that run depend on each host's OS family and whether Apache was already installed.

Step 6: Verify Apache

For Amazon Linux or RHEL:

ansible all -i inventory -m command -a "systemctl is-active httpd"

Check the service status:

ansible all -i inventory -b -m command -a "systemctl status httpd"

For Ubuntu/Debian, replace httpd with apache2.

Important: If you are using Amazon Linux 2023, the dnf module and httpd package in this playbook are appropriate.

Interview answer

What is the purpose of the when condition in Ansible?

The when condition is used for conditional task execution. Ansible runs a task only when the specified expression evaluates to true. For example, we can install httpd on Red Hat-based systems and apache2 on Debian-based systems using ansible_os_family.

3.Create ansible playbook using roles to configure LAMP stack

Configure LAMP Stack Using Ansible Roles — Step-by-Step Lab

We will create an Ansible project using roles to install and configure the LAMP stack on Linux servers.

LAMP stands for:

- L — Linux (operating system)
- A — Apache (web server)
- M — MySQL or MariaDB (database)
- P — PHP (server-side scripting language)

For your existing Ansible lab, we'll use your Amazon Linux 2023 worker nodes. We'll install Apache (httpd), MariaDB, and PHP, then deploy a PHP test page.

Step 1: Understand the role-based architecture

Ansible Control Node

Runs the main playbook

Main playbook: site.yml

Apache

Role

MariaDB

Role

PHP

Role

Managed Linux servers

Apache + MariaDB + PHP + test webpage

Each role has its own tasks and can be reused in other playbooks.

Step 2: Create the project directory

Log in to your Ansible control node and run:

mkdir -p ~/lamp-ansible

cd ~/lamp-ansible

Create the role directory structure:

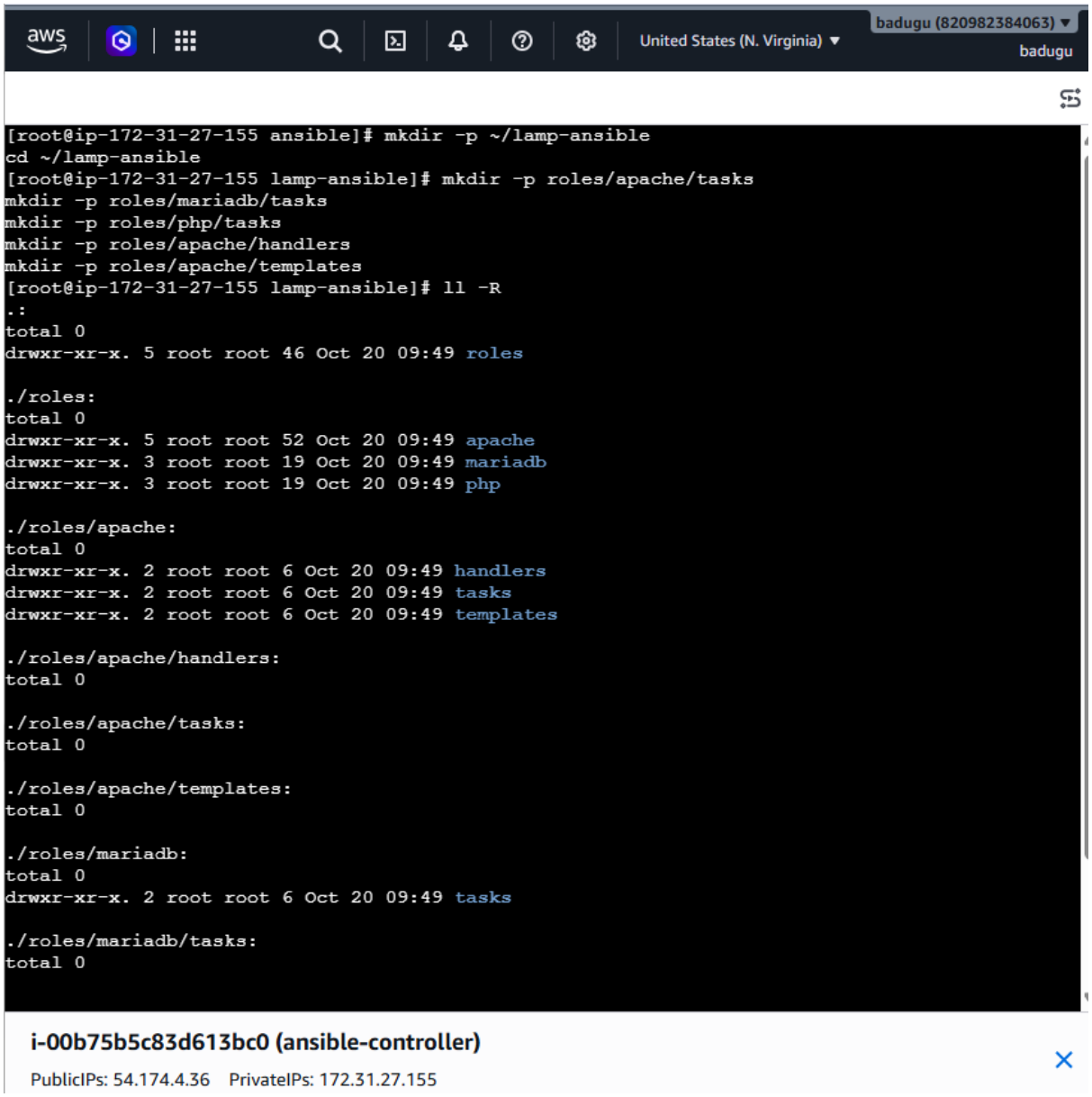
mkdir -p roles/apache/tasks

mkdir -p roles/mariadb/tasks

mkdir -p roles/php/tasks

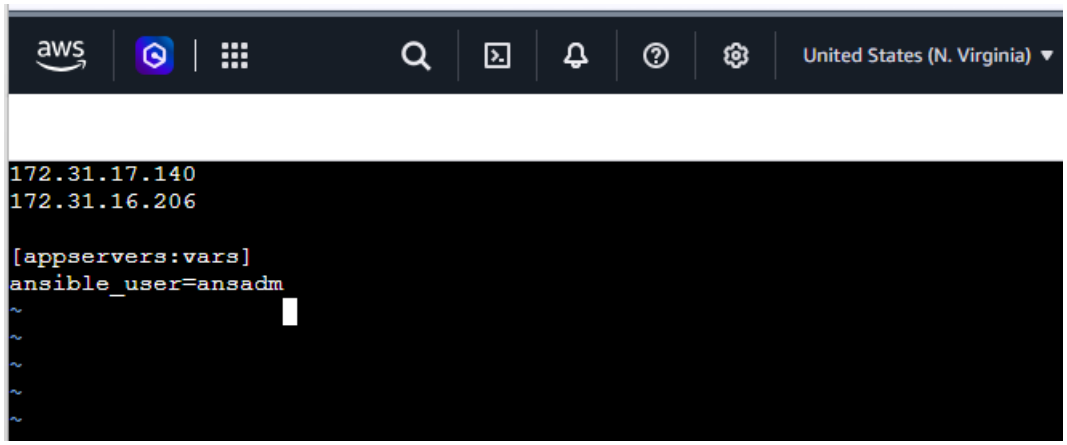
mkdir -p roles/apache/handlers

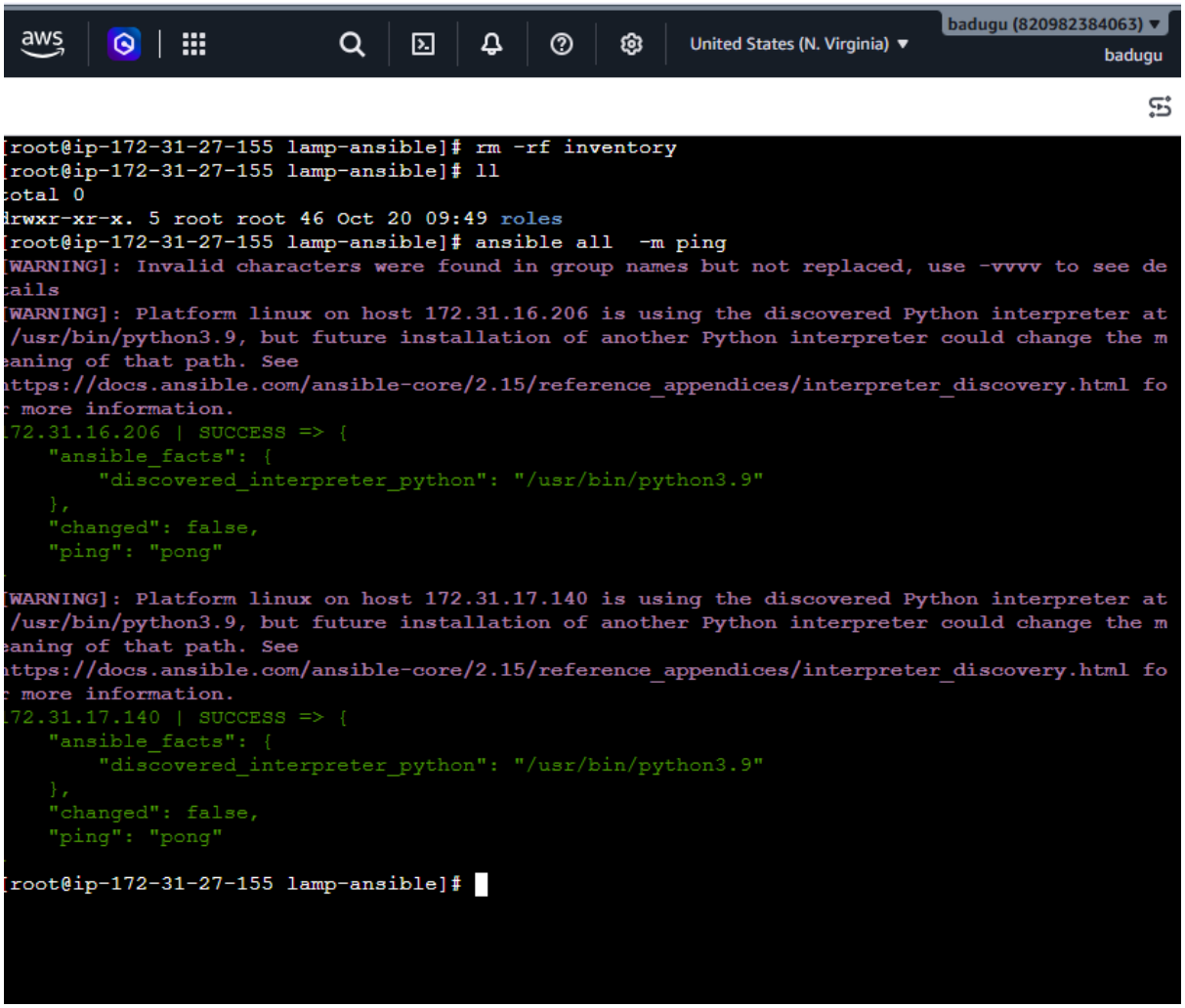
mkdir -p roles/apache/templates

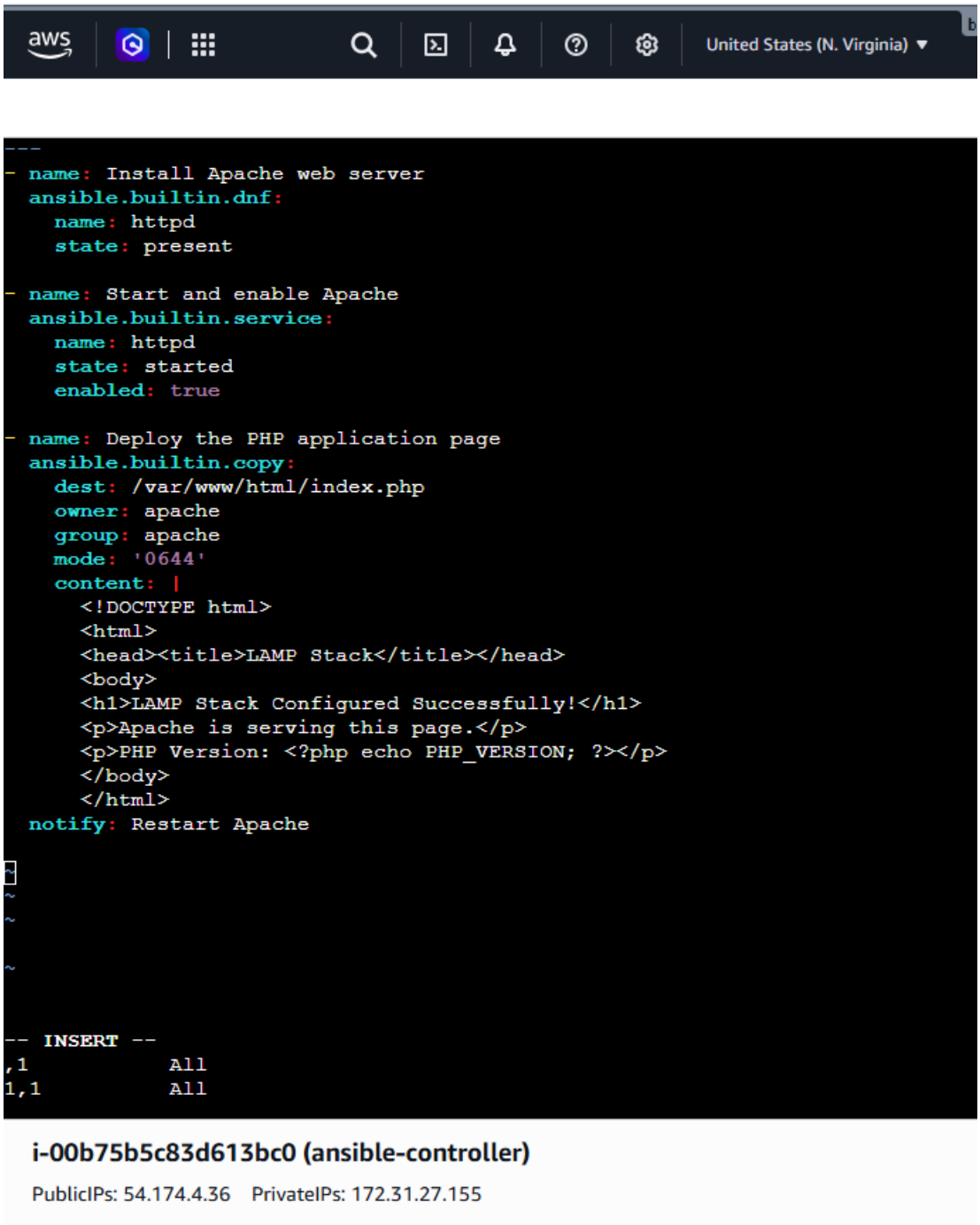


Step 3: Configure the inventory

Create the inventory file:




aws
United States (N. Virginia)
badugu (820982384063)
badugu
[root@ip-172-31-27-155 lamp-ansible]# rm -rf inventory
[root@ip-172-31-27-155 lamp-ansible]# ll
otal 0
drwxr-xr-x. 5 root root 46 Oct 20 09:49 roles
[root@ip-172-31-27-155 lamp-ansible]# ansible all -m ping
[WARNING]: Invalid characters were found in group names but not replaced, use -vvvv to see de
tails
[WARNING]: Platform linux on host 172.31.16.206 is using the discovered Python interpreter at
/usr/bin/python3.9, but future installation of another Python interpreter could change the m
eaning of that path. See
https://docs.ansible.com/ansible-core/2.15/reference_appendices/interpreter_discovery.html fo
more information.
172.31.16.206 | SUCCESS => {
"ansible_facts": {
"discovered_interpreter_python": "/usr/bin/python3.9"
},
"changed": false,
"ping": "pong"
}
[WARNING]: Platform linux on host 172.31.17.140 is using the discovered Python interpreter at
/usr/bin/python3.9, but future installation of another Python interpreter could change the m
eaning of that path. See
https://docs.ansible.com/ansible-core/2.15/reference_appendices/interpreter_discovery.html fo
more information.
172.31.17.140 | SUCCESS => {
"ansible_facts": {
"discovered_interpreter_python": "/usr/bin/python3.9"
},
"changed": false,
"ping": "pong"
}
[root@ip-172-31-27-155 lamp-ansible]#



Create the Apache handler:

vi roles/apache/handlers/main.yml

- name: Restart Apache

  ansible.builtin.service:

```
    name: httpd
    state: restarted
```

The handler restarts Apache when the application page changes. PHP-FPM will be configured in the PHP role next.

Step 5: Create the MariaDB role

Create the task file:

vi roles/mariadb/tasks/main.yml

Add:

```
---
- name: Install MariaDB server
  ansible.builtin.dnf:
    name: mariadb105-server
    state: present

- name: Start and enable MariaDB
  ansible.builtin.service:
    name: mariadb
    state: started
    enabled: true
```

This installs and starts the MariaDB database server. It does not yet create a database or configure database users.

Step 6: Create the PHP role

Create the task file:

vi roles/php/tasks/main.yml

Add:

```
---
- name: Install PHP and PHP-FPM packages
  ansible.builtin.dnf:
    name:
      - php8.4
      - php8.4-fpm
      - php8.4-mysqlnd
    state: present

- name: Start and enable PHP-FPM
  ansible.builtin.service:
    name: php-fpm
    state: started
    enabled: true
```

These package names target Amazon Linux 2023 repositories that provide PHP 8.4. Verify availability first if your configured repositories differ:

```
ansible -i inventory appservers -b -m command -a "dnf list --available 'php*'"
```

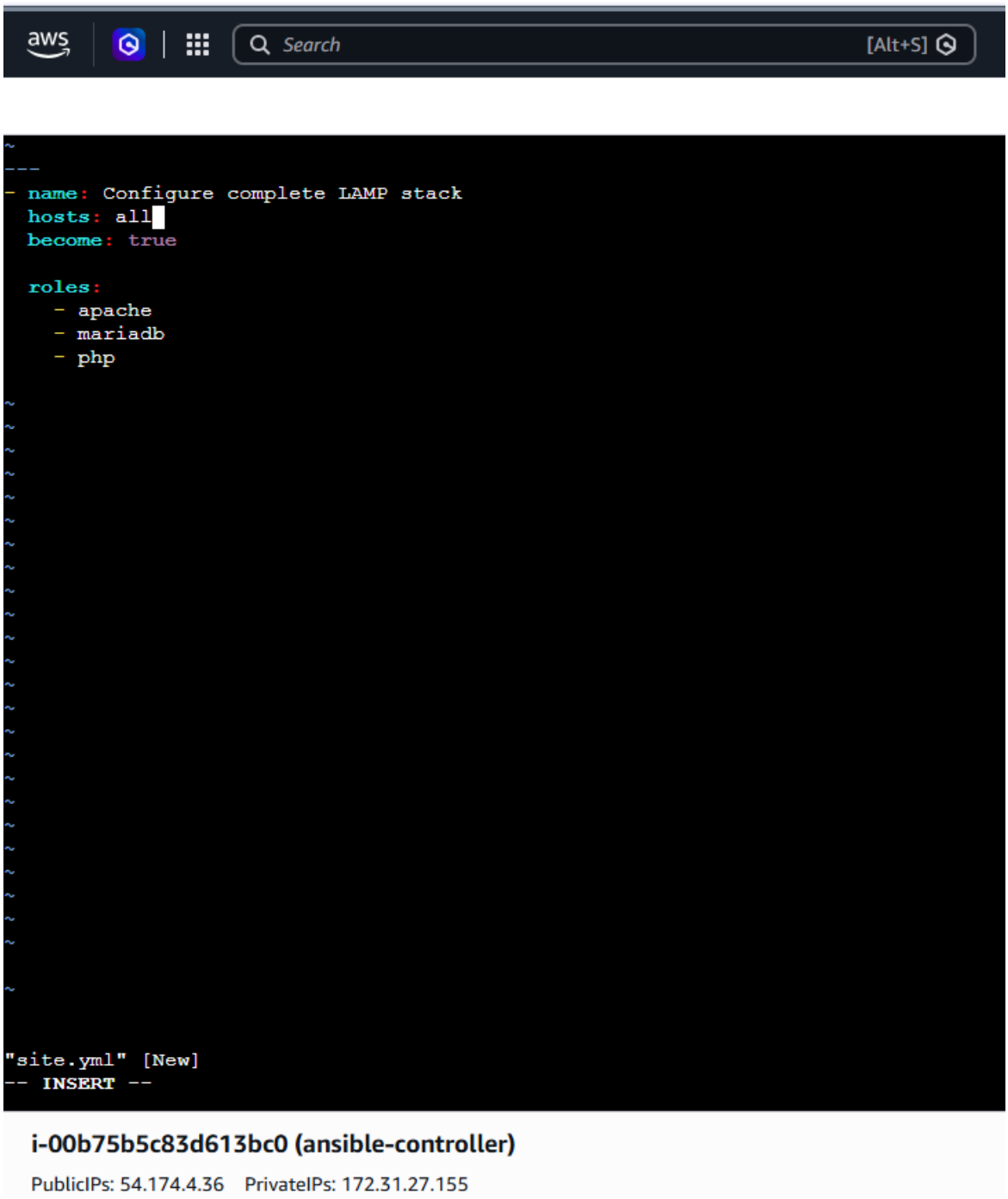
If PHP 8.4 is unavailable, use a consistent PHP version provided by your repositories and update the package names accordingly.

Step 7: Create the main playbook

Create site.yml in the project root:

```
vi site.yml
```

Add:



- name: Configure complete LAMP stack

hosts: appservers

become: true

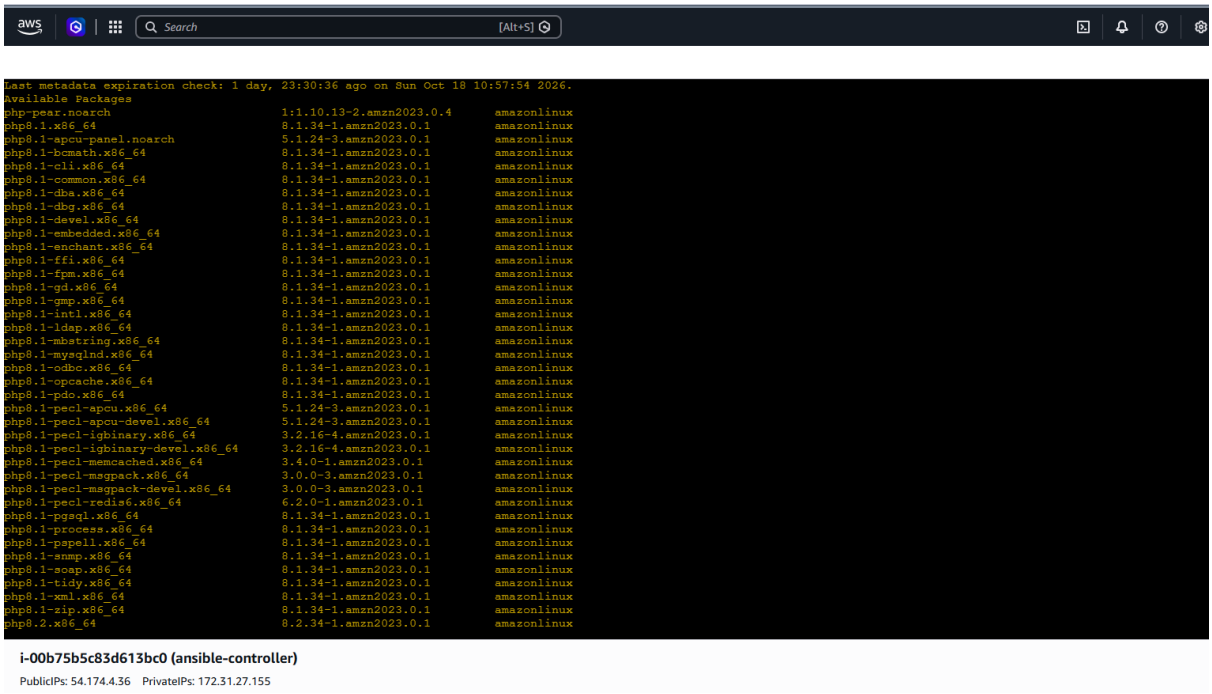
roles:

- apache

- mariadb

- php

How this works:

- hosts: appservers targets the worker servers.
- become: true allows package installation and service management.
- roles executes each role's tasks/main.yml in the listed order.
- 

Step 8: Validate the playbook

Check the YAML and role structure before running it:

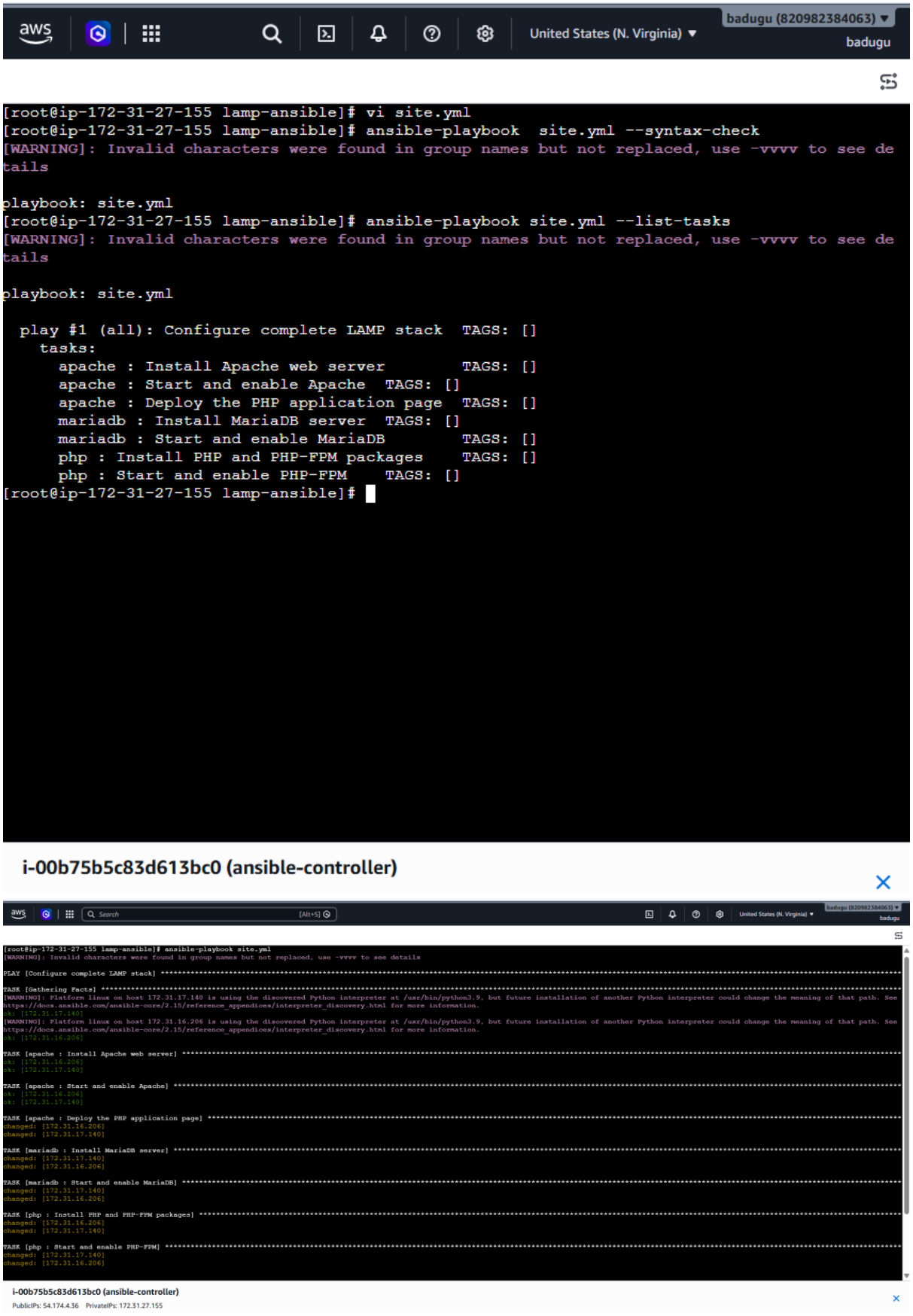
ansible-playbook -i inventory site.yml --syntax-check

Expected output:

playbook: site.yml

You can also preview the tasks:

ansible-playbook -i inventory site.yml --list-tasks



Step 9: Run the LAMP stack playbook

Execute:

ansible-playbook -i inventory site.yml

Ansible will install Apache, MariaDB, and PHP on both managed nodes, then start and enable their services.

Check the final PLAY RECAP for failed=0 on each host. If a task fails, review the error before proceeding.

Step 10: Verify the installation

Run these commands from the control node:

ansible -i inventory appservers -b -m command -a "httpd -v"

ansible -i inventory appservers -b -m command -a "mariadb --version"

ansible -i inventory appservers -b -m command -a "php -v"

Check that the services are active:

ansible -i inventory appservers -b -m command -a "systemctl is-active httpd mariadb php-fpm"

Verify that Apache can serve the PHP page locally:

ansible -i inventory appservers -b -m command -a "curl -s http://localhost/index.php"

The response should contain LAMP Stack Configured Successfully! and a PHP version number. If it returns raw PHP source code instead of executing it, troubleshoot the Apache/PHP-FPM integration before exposing the server.

Step 11: Access the application in your browser

Open the following URL using the public IP of a worker node:

http://YOUR-EC2-PUBLIC-IP/index.php

If it is not accessible, verify that:

- The EC2 security group permits inbound HTTP traffic on TCP port 80 from your intended client IP range.
- Apache is running and listening on port 80.
- The PHP-FPM integration is working.

- The instance has a reachable public IP or another suitable network route.

4.Setup ansible AWX and explore the options.

Install Ansible AWX on AWS EC2 using Minikube and AWX Operator

We'll build a separate AWX lab on Amazon Linux 2023, keeping it independent of your existing EKS cluster and Ansible control node.

By the end, you'll be able to log in to the AWX web interface, add your two worker nodes, configure SSH credentials, and run a playbook from a Job Template.

1. Architecture

Your Windows browser

Access AWX over HTTP

AWS EC2 — Amazon Linux 2023

Minikube + kubectl + AWX Operator

AWX web service and automation jobs

Worker 1

172.31.17.140

Worker 2

172.31.16.206

We'll use the AWX Operator to deploy AWX into Kubernetes. This is a learning lab, not a production deployment.

2. Create the EC2 instance

In the AWS Console, open EC2 and choose Launch instance.

Use these settings:

Setting	Recommended value
Name	awx-lab
AMI	Amazon Linux 2023

Architecture	64-bit x86
Instance type	t3.xlarge — 4 vCPU, 16 GiB RAM
Root EBS volume	50 GiB gp3
Key pair	Your existing SSH key or a new one
Public IP	Enabled, if you need browser access directly
Security group	SSH from your public IP; add AWX access later

AWX plus Kubernetes, PostgreSQL, and container images can be resource-intensive. This is a comfortable lab starting point, not a minimum guaranteed configuration. Monitor memory and disk usage.

Security group

Initially allow only:

- TCP 22 — SSH from My IP.

Later, after AWX is running, we'll allow TCP 8080 from My IP for browser access. Do not expose AWX to 0.0.0.0/0 on the internet.

Connect to the instance:

ssh -i /path/to/your-key.pem ec2-user@<EC2_PUBLIC_IP>

Replace the key path and public IP with your actual values.

3. Install Docker and required utilities

Run these commands on the new EC2 instance as ec2-user.

First, update packages and install the utilities:

sudo dnf update -y

sudo dnf install -y docker git curl tar gzip

Enable Docker:

sudo systemctl enable --now docker

sudo systemctl status docker --no-pager

Add ec2-user to the Docker group:

sudo usermod -aG docker ec2-user

Important: Docker group membership grants highly privileged access to the host. Use this only on your dedicated lab instance.

Log out and reconnect so the new group membership applies:

exit

Reconnect using your SSH command, then verify:

id

docker run --rm hello-world

If the Docker test succeeds, continue. The Minikube Docker driver runs Kubernetes in a Docker container, as described in the official Minikube Docker driver documentation.



minikube

4. Install Minikube and kubectl

Install Minikube:

curl -fSLO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64

sudo install minikube-linux-amd64 /usr/local/bin/minikube

rm -f minikube-linux-amd64

Install kubectl:

KUBECTL_VERSION=$(curl -fsSL https://dl.k8s.io/release/stable.txt)

curl -fSLO "https://dl.k8s.io/release/${KUBECTL_VERSION}/bin/linux/amd64/kubectl"

sudo install kubectl /usr/local/bin/kubectl

rm -f kubectl

Verify the tools:

minikube version

kubectl version --client

docker --version

These installation approaches follow the official Minikube installation guide.



minikube

5. Start the Kubernetes cluster

Start Minikube with Docker as the driver:

```
minikube start \
  --driver=docker \
  --cpus=4 \
  --memory=10240 \
  --disk-size=30g
```

This assigns 4 CPUs, 10 GiB RAM, and a 30 GiB Minikube disk to the cluster. Leave enough host resources for Amazon Linux and Docker.

Check the cluster:

kubectl get nodes

kubectl get pods -A

kubectl get storageclass

minikube status

Expected node status:

```
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   ...   ...
```

You should see a default storage class, commonly named standard. AWX needs persistent storage for its database. If no default storage class appears, stop here and fix storage before deploying AWX.

If Minikube fails to start, inspect:

minikube logs --problems

docker ps -a

free -h

df -h

Minikube is intended for testing and learning, not a highly available production Kubernetes deployment.



Ansible AWX Operator Documentation

+1

6. Install the AWX Operator

The AWX Operator manages the installation and lifecycle of AWX inside Kubernetes. We'll install a released version rather than tracking the development branch. Follow the official AWX Operator installation guide.



Ansible AWX Operator Documentation

+1

Create a working directory and clone the repository:

mkdir -p ~/awx-lab

cd ~/awx-lab

git clone https://github.com/ansible/awx-operator.git

cd awx-operator

Find the newest normal semantic-version tag in the cloned repository:

```
OPERATOR_TAG=$(git tag --sort=-v:refname \
  | grep -E '^[0-9]+\.[0-9]+\.[0-9]+$' \
```

```
  | head -n 1)

echo "$OPERATOR_TAG"
```

Check that a tag was returned before proceeding. Then check out that release:

```
git checkout "tags/${OPERATOR_TAG}"
```

Create the Kustomize configuration:

```
cat > kustomization.yaml <<EOF
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - github.com/ansible/awx-operator/config/default?ref=${OPERATOR_TAG}

images:
  - name: quay.io/ansible/awx-operator
    newTag: ${OPERATOR_TAG}

namespace: awx
EOF
```

Deploy the operator and its required Kubernetes resources:

```
kubectl apply -k .
```

Check the namespace and operator:

```
kubectl get namespace awx
kubectl get pods -n awx
kubectl get deployments -n awx
```

Wait until the operator pod is Running and its containers are ready. If it does not start, inspect:

kubectl describe pods -n awx

kubectl logs deployment/awx-operator-controller-manager \

  -n awx -c awx-manager --tail=100

Do not create the AWX instance until the operator has started successfully.

7. Deploy an AWX instance

Create an AWX custom resource in your working directory:

cd ~/awx-lab/awx-operator

cat > awx-demo.yml <<'EOF'

apiVersion: awx.ansible.com/v1beta1

kind: AWX

metadata:

  name: awx-demo

  namespace: awx

spec:

  service_type: ClusterIP

EOF

Apply it:

kubectl apply -f awx-demo.yml

The operator will create the AWX web application, task workers, and PostgreSQL resources. Check the progress:

kubectl get awx -n awx

kubectl get pods -n awx -w

Use Ctrl+C to stop watching; this does not stop the installation.

Once the resources are ready, check:

kubectl get pods -n awx

kubectl get services -n awx

kubectl get pvc -n awx

You should eventually see the AWX application pods ready and the PostgreSQL pod running. If any pod remains Pending, ImagePullBackOff, or CrashLoopBackOff, diagnose that before continuing.

8. Retrieve the AWX admin password

The default username is admin. Retrieve the password from the Kubernetes Secret:

```
kubectl get secret awx-demo-admin-password \
  -n awx \
  -o jsonpath='{.data.password}' | base64 --decode
echo
```

Save this password securely. Do not paste it into a public terminal log, Git repository, or chat.

If the Secret does not exist yet, wait for AWX reconciliation to finish and check the pods and operator logs again.

9. Access AWX from your Windows browser

We'll use Kubernetes port forwarding. This avoids relying on the Minikube container's internal IP being directly reachable from outside EC2.

9.1 Confirm the service name

On EC2:

kubectl get svc -n awx

You should see a service named awx-demo-service. If its name differs, use the actual service name in the next command.

9.2 Start port forwarding

Run this command in an SSH terminal and leave it running:

```
kubectl port-forward \
  --address 0.0.0.0 \
```

```
-n awx \
svc/awx-demo-service 8080:80
```

Open a second SSH session for the remaining commands. The port-forward process must remain running while you use the browser.

9.3 Update the EC2 security group

In AWS Console:

1. Go to EC2 → Instances → your awx-lab instance.
2. Open its security group.
3. Edit inbound rules.
4. Add a rule: Custom TCP, port 8080, source My IP.
5. Save the rule.

Do not open port 8080 to the entire internet.

9.4 Log in

In your Windows browser, open:

http://<EC2_PUBLIC_IP>:8080

Replace <EC2_PUBLIC_IP> with the public IPv4 address of the AWX EC2 instance.

Log in using:

- Username: admin
- Password: the value retrieved in Step 8

This is plain HTTP, suitable only for a restricted lab. For production, configure HTTPS, persistent storage, backups, and appropriately restricted network access.

10. Configure SSH access from AWX to your worker nodes

Your AWX instance runs on a separate EC2 instance. It cannot automatically use the private SSH key stored on your original Ansible control node. We'll create a dedicated key for this lab.

10.1 Create an SSH key on the AWX EC2 instance

Run as ec2-user on the AWX EC2 instance:

```
ssh-keygen -t ed25519 -N "" \
  -f ~/.ssh/awx_workers_ed25519 \
  -C awx-lab
```

Display the public key:

```
cat ~/.ssh/awx_workers_ed25519.pub
```

Copy the public key.

10.2 Authorize this key on both workers

Using the original authorized access method for each worker, add the copied public key to:

/home/ansadm/.ssh/authorized_keys

Make sure the ownership and permissions are correct on each worker:

```
sudo chown -R ansadm:ansadm /home/ansadm/.ssh
sudo chmod 700 /home/ansadm/.ssh
sudo chmod 600 /home/ansadm/.ssh/authorized_keys
```

Also confirm that each worker's security group allows SSH (TCP 22) from the AWX EC2 instance's security group. Keep this access limited to the lab instances.

If you cannot access a worker using your original key or EC2 Instance Connect, stop here and recover that access first; do not replace or overwrite an existing authorized-keys file.

10.3 Test SSH from the AWX EC2 instance

```
ssh -i ~/.ssh/awx_workers_ed25519 \
  ansadm@172.31.17.140
```

Then:

```
ssh -i ~/.ssh/awx_workers_ed25519 \
  ansadm@172.31.16.206
```

Both connections should succeed. If prompted to trust a host fingerprint, verify that you are connecting to the intended private IP before accepting it.

If either connection fails, troubleshoot SSH before configuring AWX credentials.

11. Create a GitHub project for your first AWX job

AWX Projects can retrieve playbooks from source control. For this lab, we'll create a small Git repository containing a test playbook.

11.1 Create a repository

In GitHub, create a repository named awx-lab. For the simplest setup, make it public and do not initialize it with a README. If you prefer a private repository, you'll need to configure Git credentials in AWX.

On the AWX EC2 instance:

```
mkdir -p ~/awx-test-project
```

```
cd ~/awx-test-project
```

Create ping.yml:

```
cat > ping.yml <<'EOF'
---
- name: Test AWX connectivity to worker nodes
  hosts: workers
  gather_facts: false

  tasks:
    - name: Verify Ansible connectivity
      ansible.builtin.ping:

    - name: Display remote user
      ansible.builtin.command: id
      changed_when: false
EOF
```

Initialize Git and push the playbook to your repository:

git init -b main

git add ping.yml

git config user.name "AWX Lab"

git config user.email "awx-lab@example.com"

git commit -m "Add AWX connectivity test playbook"

git remote add origin https://github.com/<YOUR_GITHUB_USERNAME>/awx-lab.git

git push -u origin main

Replace <YOUR_GITHUB_USERNAME> with your GitHub username. If Git requests authentication, use an appropriate GitHub credential; do not put a password or token directly in the remote URL.

12. Configure AWX through the web interface

Log in to AWX and perform these steps.

A. Create an inventory

Go to Resources → Inventories → Add inventory. Name it Worker Inventory and save it.

Open the inventory, choose Hosts → Add, and add these hosts:

- 172.31.17.140
- 172.31.16.206

Set each host's Variables to:

ansible_user: ansadm

Save both hosts.

B. Create a Machine credential

Go to Resources → Credentials → Add.

Use these settings:

- Name: AWX Worker SSH
- Credential Type: Machine

- Username: ansadm
- SSH Private Key: paste the contents of ~/.ssh/awx_workers_ed25519 from the AWX EC2 instance.

To display the private key locally for entry into AWX, run:

cat ~/.ssh/awx_workers_ed25519

Paste it only into the protected AWX credential field, not into GitHub or this chat. Save the credential. Leave privilege escalation disabled for this connectivity test.

C. Create a Project

Go to Resources → Projects → Add.

- Name: AWX Lab Project
- Organization: Default
- Source Control Type: Git
- Source Control URL: https://github.com/<YOUR_GITHUB_USERNAME>/awx-lab.git
- Source Control Branch/Tag/Commit: main

Leave the source-control credential blank if the repository is public. Save the project and wait for its sync to complete.

D. Create a Job Template

Go to Resources → Templates → Add → Add job template.

Set:

- Name: AWX Worker Connectivity Test
- Job Type: Run
- Inventory: Worker Inventory
- Project: AWX Lab Project
- Playbook: ping.yml
- Credentials: AWX Worker SSH

Save the template.

The precise placement of navigation options can vary slightly by AWX version.

13. Run your first job

Open the AWX Worker Connectivity Test Job Template and click Launch.

The job should run the ping module and then display the remote user identity on both workers.

Expected playbook output will resemble:

TASK [Verify Ansible connectivity]

ok: [172.31.17.140]

ok: [172.31.16.206]

TASK [Display remote user]

ok: [172.31.17.140]

ok: [172.31.16.206]

PLAY RECAP

172.31.17.140 : ok=2 changed=0 failed=0

172.31.16.206 : ok=2 changed=0 failed=0

If the job fails, inspect the job's Details and Output tabs.

- Permission denied (publickey): verify the Machine credential and each worker's authorized_keys.
- UNREACHABLE: verify private-IP connectivity and security group rules.
- Project sync failure: verify the repository URL, branch, and repository access.
- Pending pods: inspect kubectl describe pods -n awx and check available memory and storage.

14. Explore the other AWX features

Once the first job succeeds, explore these features:

AWX feature	Practice exercise
Job Templates	Run the same playbook repeatedly
Schedules	Schedule a harmless test job
Surveys	Add a user input such as an environment name
Workflows	Chain a validation job to a deployment job
Credentials	Store SSH keys and privilege-escalation credentials
RBAC	Create a user with restricted permissions
Job output	Investigate failed tasks and review logs
Notifications	Configure a notification destination
Projects	Sync updated playbooks from Git
Backups	Learn the AWX backup and restore process

15. Stop the lab when you're finished

To stop the Kubernetes cluster while preserving its state:

```
minikube stop
```

You can then stop the EC2 instance in the AWS Console. You will still pay for its EBS storage while it is stopped. When you start it again, its public IPv4 address may change unless you use an Elastic IP. Start Minikube again with:

```
minikube start
```

Remember to restart the kubectl port-forward command and update your browser URL if the public IP changes.